

Programowanie Obiektowe

Projekt backendowy w C# - TicketFlow API

Temat projektu

Backendowy system obsługi zgłoszeń helpdesk. Program pozwala tworzyć klientów, pracowników wsparcia i zgłoszenia, przypisywać zgłoszenia do osób, zmieniać statusy, wyliczać przeterminowane sprawy oraz generować raporty.

Informacja	Wartość
Autor	Michał Matusiewicz 82957
Technologia	C# 12, .NET 8, ASP.NET Core Web API
Charakter	Projekt obiektowy, backendowy, bez frontendu
Zakres	Klasy, konstruktory, dziedziczenie, polimorfizm, interfejsy, typy ogólne, zdarzenia, operatory, async, refleksja

Spis treści

1. Cel i opis programu	3
2. Struktura projektu	4
3. API i zakres funkcjonalny	5
4. Elementy obiektowości	6
5. Spełnienie kryteriów projektu	7
6. Kluczowe fragmenty kodu	8
7. Podsumowanie	9

Uwaga redakcyjna

Dokument zawiera wyłącznie informacje potrzebne do oceny projektu: opis działania, strukturę, zakres API, zastosowane mechanizmy obiektowe, fragmenty kodu i krótkie uzasadnienia projektowe.

1. Cel i opis programu

TicketFlow API jest backendowym systemem do obsługi zgłoszeń helpdesk. Program przechowuje klientów, pracowników wsparcia oraz zgłoszenia, a następnie udostępnia operacje zarządzania przez endpointy HTTP.

Najprostszy opis działania

System działa jak rejestr spraw. Klient tworzy zgłoszenie, pracownik wsparcia może je przejąć, a aplikacja pilnuje statusu, priorytetu, komentarzy i limitu SLA.

Główne funkcje

Funkcja	Opis
Zgłoszenia	Tworzenie, pobieranie, przypisywanie, zmiana statusu i dodawanie komentarzy.
Klienci i agenci	Obsługa klientów oraz pracowników wsparcia.
SLA	Sprawdzanie, które zgłoszenia przekroczyły czas obsługi.
Raportowanie	Eksport listy zgłoszeń do formatu CSV lub tekstowego.
Metadane	Automatyczny opis klas domenowych za pomocą refleksji.

2. Struktura projektu

Projekt jest podzielony na warstwy. Dzięki temu kontrolery nie zawierają całej logiki, a klasy domenowe nie są zależne od HTTP.

Układ katalogów

```
TicketFlowProject/  
|-- TicketFlow.sln  
|-- README.md  
|-- docs/  
| |-- Dokumentacja_TicketFlow_API_estetyczna_poprawiona.pdf  
|-- src/TicketFlow.Api/  
| |-- Api/Controllers/ endpointy HTTP  
| |-- Api/Requests/ modele danych wejściowych  
| |-- Application/ usługi, repozytoria, raporty, powiadomienia  
| |-- Domain/ najważniejsze klasy obiektowe  
| |-- Infrastructure/Seed/ dane startowe  
| |-- Program.cs konfiguracja aplikacji
```

Najważniejsze klasy

Klasa	Rola w projekcie
Ticket	Bazowa klasa zgłoszenia. Ma tytuł, opis, priorytet, komentarze, SLA i status.
IncidentTicket	Specjalny rodzaj zgłoszenia awarii. Inaczej wylicza koszt obsługi.
ServiceRequestTicket	Zgłoszenie prośby o usługę, np. utworzenie konta.
Customer	Klient, który może tworzyć zgłoszenia.
SupportAgent	Pracownik wsparcia, do którego można przypisać zgłoszenie.
TicketService	Główna usługa aplikacyjna obsługująca przypadki użycia.
InMemoryRepository<T>	Repozytorium ogólne przechowujące obiekty w pamięci aplikacji.

3. API i zakres funkcjonalny

Aplikacja udostępnia REST API. Kontrolery przyjmują żądania HTTP, przekazują je do usług aplikacyjnych i zwracają wynik w formacie JSON albo jako plik CSV.

Najważniejsze endpointy

Metoda	Adres	Co robi
GET	/api/tickets	Zwraca wszystkie zgłoszenia.
POST	/api/tickets	Tworzy nowe zgłoszenie.
PUT	/api/tickets/{id}/assign	Przypisuje zgłoszenie do pracownika.
PUT	/api/tickets/{id}/status	Zmienia status zgłoszenia.
POST	/api/tickets/{id}/comments	Dodaje komentarz do zgłoszenia.
GET	/api/tickets/overdue	Zwraca zgłoszenia po terminie SLA.
GET	/api/reports/tickets.csv	Generuje raport CSV.
GET	/api/metadata/domain	Zwraca metadane klas domenowych.

Przykład danych wejściowych

POST /api/tickets

```
{
  "title": "Brak dostępu do panelu",
  "description": "Użytkownik nie może zalogować się do systemu.",
  "customerId": "guid-klienta",
  "priority": "High",
  "kind": "Incident"
}
```

4. Elementy obiektowości

Poniższa tabela pokazuje wymagane mechanizmy obiektowości i konkretne miejsca ich użycia w projekcie. Każdy element został zastosowany w funkcji systemu, a nie wyłącznie jako sztuczny przykład.

Element	Gdzie w kodzie	Po co
Klasy	Ticket, Customer, SupportAgent, TicketService	Opisują rzeczy występujące w systemie: zgłoszenie, klienta, pracownika i usługę.
Konstruktory	Ticket(...), Customer(...), SupportAgent(...)	Tworzą obiekty od razu w poprawnym stanie.
Właściwości / indeksatory	FullName, Status, Priority, ticket[0], customer[0]	Udostępniają dane w kontrolowany sposób i upraszczają dostęp do kolekcji.
Statyczne	SlaPolicy.GetLimitFor(...), Money.Zero(...)	Dla reguł i wartości pomocniczych niezależnych od konkretnego obiektu.
Dziedziczenie	Ticket -> IncidentTicket / ServiceRequestTicket	Wspólne cechy są w klasie bazowej, różnice w klasach potomnych.
Polimorfizm	EstimateCost()	Ten sam wywołany mechanizm daje inny wynik zależnie od typu zgłoszenia.
Interfejsy / abstrakcja	IRepository<T>, INotificationChannel, IReportExporter	Oddzielają kontrakt od implementacji.
Typy ogólne / kolekcje	InMemoryRepository<T>, OperationResult<T>, List<T>, Dictionary<TKey,TValue>	Pozwalają pisać uniwersalny i typowany kod.
Delegacje / zdarzenia	TicketChangedHandler, event TicketChanged	Pozwalają reagować na zmianę zgłoszenia bez wiązania modułów.
Przeciążanie operatorów	Money + Money, Money * decimal, SlaTime > SlaTime	Upraszcza zapis działań na obiektach wartości.
Programowanie asynchroniczne	CreateAsync, AssignAsync, ExportAsync, SendAsync	Dopasowuje backend do operacji I/O i API.
Refleksja	ReflectionMetadataService	Automatycznie odczytuje informacje o klasach domenowych.

5. Spełnienie kryteriów projektu

Projekt spełnia wymagania z listy zajęciowej. Zakres projektu jest backendowy i obejmuje modele domenowe, logikę aplikacyjną, kontrolery HTTP, repozytoria, raporty, powiadomienia, obsługę SLA oraz wymagane mechanizmy obiektowości.

Kryterium	Realizacja w projekcie
Zakres backendowy	Modele domenowe, kontrolery HTTP, usługi aplikacyjne, repozytoria w pamięci, raport CSV i metadane klas.
Obsługa zgłoszeń	Tworzenie, przypisywanie, zmiana statusu, komentarze i wyszukiwanie zgłoszeń po terminie SLA.
Mechanizmy obiektowe	W kodzie występują wszystkie elementy z listy: dziedziczenie, polimorfizm, interfejsy, typy ogólne, zdarzenia, operatory, refleksja i async.
Czytelna struktura	Kod podzielono na warstwy Api, Application, Domain i Infrastructure, co oddziela HTTP od logiki domenowej.
Rozszerzalność	Interfejsy pozwalają wymienić repozytorium, kanał powiadomień albo format raportu bez przepisywania kontrolerów.

Uwagi projektowe

Aplikacja nie zawiera frontendu ani stałej bazy danych, ponieważ celem projektu jest pokazanie mechanizmów C# i OOP w warstwie backendowej. Dane przykładowe są przechowywane w pamięci, aby uprościć uruchomienie i ocenę kodu.

6. Kluczowe fragmenty kodu

Poniżej znajdują się najważniejsze fragmenty kodu wraz z krótkim uzasadnieniem. Opisy są celowo krótkie i odnoszą się tylko do decyzji projektowych.

Dziedziczenie i polimorfizm

```
public abstract class Ticket : Entity
{
    public abstract TicketKind Kind { get; }
    public abstract Money EstimateCost();
}

public sealed class IncidentTicket : Ticket
{
    public override TicketKind Kind => TicketKind.Incident;
    public override Money EstimateCost()
    {
        return SlaPolicy.GetBasePriceFor(Priority) * 1.35m;
    }
}
```

Dlaczego użyto tego rozwiązania?

Wybrano klasę abstrakcyjną, bo wszystkie zgłoszenia mają wspólne pola, ale różnią się typem i wyceną. Dzięki polimorfizmowi kod może wywołać EstimateCost() bez sprawdzania rodzaju zgłoszenia instrukcjami if.

Delegacja i zdarzenie

```
public delegate Task TicketChangedHandler(
    object sender,
    TicketChangedEventArgs eventArgs,
    CancellationToken cancellationToken = default
);

public event TicketChangedHandler? TicketChanged;
```

Dlaczego użyto tego rozwiązania?

Zdarzenie pasuje do sytuacji, w której zmiana zgłoszenia powinna uruchomić dodatkowe reakcje, np. powiadomienie. Usługa zgłoszeń nie musi znać szczegółów kanałów powiadomień.

6. Kluczowe fragmenty kodu - cd.

Typ ogólny i indeksator

```
public sealed class InMemoryRepository<T> : IRepository<T> where T : Entity
{
    private readonly Dictionary<Guid, T> _items = new();

    public T? this[Guid id] =>
        _items.TryGetValue(id, out var item) ? item : null;
}
```

Dlaczego użyto tego rozwiązania?

Typ ogólny pozwala użyć jednego repozytorium dla klientów, agentów i zgłoszeń. Indeksator daje krótki, czytelny dostęp do elementu po identyfikatorze.

Przeciążanie operatorów i refleksja

```
public static Money operator +(Money left, Money right)
{
    if (left.Currency != right.Currency)
        throw new InvalidOperationException("Różne waluty.");

    return new Money(left.Amount + right.Amount, left.Currency);
}

var properties = type
    .GetProperties(BindingFlags.Public | BindingFlags.Instance)
    .Select(property => property.Name)
    .ToList();
```

Dlaczego użyto tego rozwiązania?

Operatory sprawiają, że obiekty wartości Money i SlaTime zachowują się naturalnie przy dodawaniu i porównywaniu. Refleksja została użyta tylko tam, gdzie faktycznie ma sens: do automatycznego opisu klas domenowych.

Podsumowanie

TicketFlow API jest projektem backendowym pokazującym praktyczne zastosowanie obiektowości w C#. Projekt nie ogranicza się do prostego CRUD, ponieważ zawiera zdarzenia, polimorfizm, interfejsy, typy ogólne, przeciążone operatory, programowanie asynchroniczne i refleksję.